

Kế hoạch triển khai MealFit

Đây là tài liệu kế hoạch chi tiết để triển khai toàn bộ chức năng cho ứng dụng bán hàng **MealFit** dựa trên nền tảng RAG HUB. Tài liệu bao gồm mô hình dữ liệu, yêu cầu UI/UX, quy trình seed dữ liệu từ file Excel, phân quyền người dùng và các bước triển khai cụ thể.

1. Mục tiêu

- Quản lý món và combo:** Lưu trữ danh sách món ăn, phiên bản theo trọng lượng (100g, 150g, 200g, ...), giá vốn, giá bán, lợi nhuận và tỉ lệ lãi.
- Quản lý đơn hàng:** Tạo mới đơn hàng, gán khách hàng và nhân viên, tính tổng số lượng, tổng tiền, giảm giá, phí ship và trạng thái thanh toán.
- Quản lý chi tiết đơn hàng:** Lưu các món/combo trong từng đơn cùng trọng lượng, số lượng, đơn giá và lợi nhuận.
- Quản lý khách hàng và nhân viên:** Danh sách khách hàng với thông tin liên lạc; danh sách nhân viên với phân quyền **admin** hoặc **nhân viên**.
- Báo cáo doanh thu và lợi nhuận:** Tổng hợp doanh thu, vốn, lãi theo khoảng thời gian, theo khách hàng, theo món hoặc combo.
- UI/UX chuyên nghiệp:** Giao diện đẹp, responsive ở các kích thước 320–430 px (mobile), 768–1024 px (tablet) và 1280–1920 px (desktop), không tràn UI hay xuất hiện thanh cuộn ngang.
- Hiệu ứng loading:** Màn hình chờ khi tải trang, hiệu ứng loading khi submit form, nhấn nút.
- Drawer + Toast + Validation:** Form CRUD hiển thị bằng drawer trượt từ phải; validation phía client; thông báo toast (success/error/info) ở góc trên phải tự ẩn sau 4 s; đóng drawer bằng ESC hoặc click ra ngoài.
- Nhập dữ liệu từ file Excel:** Viết script seed để đọc file Excel (đã cung cấp) và chèn dữ liệu vào các bảng mới của Supabase.
- Trang landing công khai:** Trang giới thiệu sản phẩm, trình bày thực đơn và combo dưới dạng landing page, không cần đăng nhập.

2. Mô hình dữ liệu

Bổ sung các bảng riêng cho sản phẩm MealFit trong **Supabase** (xem file `supabase/migrations/20260613160000_mealfit.sql` để biết chi tiết):

Bảng	Mục đích
<code>mealfit_products</code>	Lưu từng món ăn theo mã (<code>code</code>), tên, danh mục, trọng lượng (<code>weight</code>), giá vốn, giá bán, lợi nhuận, tỉ lệ lãi
<code>mealfit_combos</code>	Lưu các combo với mã, tên, mô tả, giá bán, giá vốn, lợi nhuận, tỉ lệ lãi
<code>mealfit_combo_products</code>	Liên kết combo với các món con; mỗi hàng gồm <code>combo_id</code> , <code>product_id</code> , số lượng hoặc trọng lượng
<code>mealfit_customers</code>	Danh sách khách hàng: mã, tên, số điện thoại, địa chỉ, ghi chú

Bảng	Mục đích
<code>mealfit_employees</code>	Danh sách nhân viên với phân quyền (<code>role</code> = <code>admin</code> / <code>staff</code>), tên, email, mật khẩu mã hoá
<code>mealfit_orders</code>	Đơn hàng: mã đơn, khách hàng, nhân viên tạo, ngày, tổng số lượng, tổng tiền, tổng vốn, giảm giá, phí ship, trạng thái
<code>mealfit_order_items</code>	Chi tiết đơn: thuộc đơn nào, món hoặc combo nào, số lượng, trọng lượng, giá vốn, giá bán, lợi nhuận

Ngoài ra có thể mở rộng thêm các bảng quản lý tồn kho (`mealfit_inventory`), nhập/xuất kho và bảng danh mục nguyên liệu nếu cần.

3. Quy trình seed dữ liệu

1. Đặt file Excel gốc (ví dụ `data/mealfit.xlsx`) vào thư mục `scripts/data/`.
2. Viết script TypeScript/Node (`scripts/seed/mealfit.ts`) sử dụng thư viện `xlsx` để đọc các sheet `_Thuc_don`, `Bang_gia`, `Combo_tong_hop`, `Combo_chi_tiet`, `Khach_hang`, `Don_hang`, `Chi_tiet_don_hang`.
3. Chuyển đổi dữ liệu thành cấu trúc phù hợp với các bảng ở mục 2.
4. Sử dụng SDK `@supabase/supabase-js` với `SUPABASE_SERVICE_ROLE_KEY` để gọi `supabase.from(<table>).upsert()` dữ liệu; đảm bảo unique theo `code` để idempotent.
5. Seed nhân viên mặc định: tạo tài khoản admin và một số tài khoản nhân viên với mật khẩu tạm thời.
6. Lưu script vào repo và chạy bằng lệnh: `TS_NODE_TRANSPILE_ONLY=true ts-node scripts/seed/mealfit.ts` sau khi đã cấu hình `.env`.

4. Yêu cầu UI/UX

- **Typography:** Body sử dụng font `"Avenir Next", "Segoe UI", sans-serif`, cỡ cơ bản 14 px. Tiêu đề `h1` 21 px, phần thống kê 22 px, `h3` 15 px.
- **Nút và icon:** Thu nhỏ padding, font và radius của các nút; thêm icon cho tất cả nút (quay lại, xác nhận, hoàn tất, tải template, nạp, lưu, hủy, tạo, đăng nhập, sửa, xóa).
- **Responsive:** Sử dụng grid/flex để bố cục chia cột hợp lý. Tránh đặt giá trị cố định gây tràn; dùng media queries Tailwind/Next.js responsive: `sm` (≥ 640 px), `md` (≥ 768 px), `lg` (≥ 1024 px), `xl` (≥ 1280 px), `2xl` (≥ 1536 px).
- **Loading states:** Hiển thị spinner hoặc skeleton cho bảng khi đang tải; disable nút trong khi gửi form.
- **Animation nền:** Thêm một hiệu ứng animated SVG hoặc CSS gradient chạy nền (nhẹ, không che nội dung) để tăng trải nghiệm.
- **Drawer & Toast:** Tạo component `Drawer` và `Toast` tái sử dụng. Drawer trượt từ phải, overlay có thể click ra ngoài hoặc nhấn `Esc` để đóng. Toast hiển thị ở góc trên phải, có trạng thái success/error/info, tự ẩn sau 4 s.
- **Form validation:** Sử dụng `react-hook-form` kết hợp `zod` để validate. Đánh dấu các trường bắt buộc, hiển thị lỗi rõ ràng.
- **Import template:** Cung cấp nút tải file mẫu Excel. Khi người dùng upload file, hiển thị bảng lỗi (dòng/cột/giá trị/lỗi) và chặn import nếu vẫn còn lỗi. Dùng `useParams` để lấy tham số đường dẫn và hiển thị toast phù hợp.

5. Phân quyền và xác thực

- **Admin:** Toàn quyền quản lý sản phẩm, combo, khách hàng, đơn hàng, nhân viên và báo cáo. Có thể xem/nhập xuất kho.
- **Nhân viên:** Chỉ được tạo đơn hàng, xem danh sách món và combo, xem báo cáo cá nhân; không được xoá hoặc chỉnh sửa sản phẩm.
- Sử dụng Supabase Auth: thêm nhân viên qua bảng `mealfit_employees` và đồng bộ với `auth.users` nếu cần. Có thể dùng Google OAuth cho khách hàng đăng nhập xem đơn hàng của họ (tùy mở rộng).

6. Các bước triển khai

1. **Thiết kế dữ liệu:** Xem lại file Excel, xác định mọi thuộc tính cần lưu. Viết migration SQL (đính kèm file `20260613160000_mealfit.sql`).
2. **Seed dữ liệu:** Viết script `mealfit.ts`, đọc Excel và import vào Supabase. Chạy kiểm thử local.
3. **Tạo API layer:** Tạo route handler `/api/mealfit/*` cho CRUD sản phẩm, combo, đơn hàng. Sử dụng Prisma hoặc trực tiếp Supabase client tùy stack.
4. **Xây dựng UI:** Tạo các page `/mealfit/products`, `/mealfit/combo`, `/mealfit/orders`, `/mealfit/customers`, `/mealfit/reports` và `/mealfit/landing`. Sử dụng layout của RAG HUB, áp dụng responsive.
5. **Form và Drawer:** Tạo component chung cho form (sản phẩm, combo, đơn hàng). Áp dụng validation, toast.
6. **Phân quyền UI:** Ẩn/hiện nút theo vai trò.
7. **Testing:** Chạy `npm run typecheck` và `npm run build` để đảm bảo không lỗi. Viết unit test với `vitest` cho các hàm tính lợi nhuận.
8. **Cập nhật tài liệu:** Cập nhật `AGENTS.md` và `CLAUDE.md` với hướng dẫn mới; thêm entry vào `docs/commit_prompt_map.md` cho commit.
9. **Triển khai:** Đẩy migration và seed lên Supabase (`supabase db push`), deploy Next.js app, kiểm tra thực tế.
10. **Iterate:** Thu thập phản hồi người dùng, điều chỉnh UI/UX và chức năng khi cần.

Tài liệu này là điểm khởi đầu cho quá trình xây dựng một hệ thống MealFit hoàn chỉnh dựa trên RAG HUB. Các bước có thể được điều chỉnh tùy theo yêu cầu thực tế và phản hồi của người dùng.
